
Principles of Programming Languages:

Problem Sheet 2 – Memory and monads

Mike Spivey, Michaelmas Term 2012.

- 1 After the definitions,

```
val x = new();;  
val y = x;;
```

the names x and y ought to denote the same storage location, and after the definitions,

```
val x = new();;  
val y = new();;
```

they should denote different locations, and the values `<address 0>` and `<address 1>` displayed by the FunMem interpreter seem to confirm it. Devise an experiment, in the form of an expression that gives different integer or boolean values after the two pairs of declarations, to demonstrate that this is the case, without directly comparing the addresses of x and y .

- 2 In FunMem, is it possible for the value of a variable to be another variable? Give an example, or explain why it can't happen. How can we construct a cons cell that has a variable as its tail? Show how to use this idea to construct cyclic lists in FunMem.

- 3 Many Pascal-like languages have a repeat loop that we might give the concrete syntax

```
repeat expr until expr
```

The expression `repeat  $e_1$  until  $e_2$` is evaluated by repeatedly evaluating e_1 (and discarding its value) followed by e_2 (whose value must be a Boolean), until the last evaluation of e_2 yields `true`. Show how to add this construct to FunMem.

- 4 In the language FunMem, we added assignment as a language construct, extending the abstract syntax of Fun to do so. By defining a primitive `put` such that `put( $x$ ,  $y$ )` is equivalent to $x := y$, show that we could in fact have made assignment a primitive.

- 5 Show that the three monad laws are satisfied by the operations of the monad of memory.

2 Principles of Programming Languages: Problem sheet 2

6 Define a monad M that supports both memory and output, together with operations

$get :: Location \rightarrow M\ Value$
 $put :: Location \rightarrow Value \rightarrow M\ ()$
 $new :: M\ Location$
 $output :: String \rightarrow M\ ()$

- 7 (a) In the basic language Fun, we cannot tell whether the actual parameters of a function are evaluated from left to right or from right to left, nor whether the function itself is evaluated (to obtain the function object that is applied to the arguments) before or after the actual parameters. Why is this? Write a program in Fun with memory that reveal these secrets about the interpreter.
- (b) Show how to modify the monadic interpreter so that actual parameters are evaluated from right to left, and the function itself is evaluated after the parameters.
- 8 (a) Using the monad laws, show that the programs $e_1; (e_2; e_3)$ and $(e_1; e_2); e_3$ are equivalent, and that $nil; e_1$ is equivalent to e_1 . Why is $e_1; nil$ not equivalent to e_1 in general?
- (b) Show that the expression $e_1; e_2$ can be regarded as syntactic sugar for `let val  $x = e_1$  in  $e_2$` , where x is an identifier that does not occur in the source program.

9 In order to put the interpreter for Fun into monadic form, we replaced the use of *map* for evaluating lists of arguments with a function *evalargs*, defined as follows.

$evalargs :: [Expr] \rightarrow Env \rightarrow M\ [Value]$
 $evalargs\ []\ env = result\ []$
 $evalargs\ (e : es)\ env =$
 $eval\ e\ env \triangleright (\lambda v \rightarrow evalargs\ es\ env \triangleright (\lambda vs \rightarrow result\ (v : vs)))$

Define a polymorphic function *mapm* with type,

$mapm :: (a \rightarrow M\ b) \rightarrow [a] \rightarrow M\ [b]$,

such that $evalargs\ aps\ env$ can be replaced by

$mapm\ (\lambda e \rightarrow eval\ e\ env)\ aps$

10 Suppose that $result :: \alpha \rightarrow M\ \alpha$ and $(\triangleright) :: M\ \alpha \rightarrow (\alpha \rightarrow M\ \beta) \rightarrow M\ \beta$ satisfy the three monad laws. Show that it is possible to define polymorphic functions $mmap :: (\alpha \rightarrow \beta) \rightarrow (M\ \alpha \rightarrow M\ \beta)$ and $join :: M\ (M\ \alpha) \rightarrow M\ \alpha$ in such a way that $xm \triangleright f = join\ (mmap\ f\ xm)$. Show that *mmap* and *join* satisfy the following laws:

$mmap\ id = id$
 $mmap\ (g \cdot f) = mmap\ g \cdot mmap\ f$
 $mmap\ f \cdot result = result \cdot f$
 $mmap\ f \cdot join = join \cdot mmap\ (mmap\ f)$

Show in addition that the following laws hold:

$$join \cdot join = join \cdot map \, join$$

$$join \cdot result = id$$

$$join \cdot mmap \, result = id$$

Conversely, if *result*, *mmap* and *join* satisfy all seven of these laws and $\triangleright$ is defined by the equation $xm \triangleright f = join \, (mmap \, f \, xm)$, then *result* and *bind* together satisfy the three monad laws.¹

¹ In the language of Category Theory, the first four laws stated above express the facts that *mmap* is (the arrow part of) a 'functor', and that *result* and *join* are 'natural transformations'. This exercise shows that our definition of the concept of a monad is equivalent to a formulation in terms of a functor and two natural transformations that together satisfy the three laws shown, which are expressible by saying that certain 'diagrams commute'. Our formulation is more convenient for programming, and the other is more convenient for point-free algebraic manipulations.